P2880R0
Algorithm-like vs std::simd based RNG API

Published Proposal, 2023-05-18

This version:
http://wg21.link/P2880R0

Authors:
Ilya Burylov
Ruslan Arutyunyan (Intel)
Andrey Nikolaev
Alina Elizarova (Intel)
Pavel Dyakov (Intel)

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The paper compares the approaches of high-level (algorithm-like) vector API for random number generation (RNG) vs low-level (`std::simd` based) one. The main goal of this paper is to show the need for co-existance of both API levels, support the [P1068R7] paper and pave the way for `std::simd` in RNG.

Table of Contents

§ 1. Introduction

There is a [P1068R7] paper on the flight, which discusses a higher level API to enable (RNG) in batches. The main purpose of the new API is to enable vectorization under the hood in the implementation. Given some of us live in anticipation of final `std::simd` [P1928R3] standardization, a natural questions arise:

- Do we need a dedicated high-level API for RNG vectorization?
- Can `std::simd`-based low-level API with some generic mechanism for making it high level be sufficient?

§ 2. Example

When does RNG performance matter? An algorithm should consume many random numbers. Additional computation on top of generation should be relatively lightweight. The "European options pricing" benchmark can show the purpose:

```
std::mt19937 engine(777);
std::normal_distribution distribution(0., 1.);

double v0 = 0, v1 = 0;
for (std::size_t p = 0; p < npath; ++p) { //e.g., npath=1,000,000
    double rand = distribution(engine);
    double res = std::max(0., S * exp(sqrt(T) * vol * rand + T * mu) - X);
    v0 += res;
    v1 += res * res;
```

```
}
result     = v0 / npath;
confidence = v1 / npath;
```

One should have millions of iterations of the loop to get an accurate estimation of the price: many random numbers will be consumed with a handful of operations on top.

The loop will not be auto-vectorized by compilers because of RNG - lack of performance out of the box. Production code is tending to be more complicated: there can be more than one random number per computational block with potentially different distributions, random number paths may need to be correlated, etc. but the "European option pricing" benchmark is good enough to show different approaches.

§ 2.1. High-level API with [P1068R7]

Let’s consider the straightforward example

```
std::mt19937 engine(777);
std::normal_distribution distribution(0., 1.);
std::array<double, npath> rand; // npath=1,000,000 -> sizeof(rand)=8 MB

std::ranges::generate_random(rand, engine, distribution);

double v0 = 0, v1 = 0;

for(std::size_t p = 0; p < npath; ++p) {
    double res = std::max(0., S * exp(sqrt(T) * vol * rand[p] + T * mu) - X
    v0 += res;
    v1 += res * res;
}

result     = v0 / npath;
confidence = v1 / npath;
```

We got vectorization both in `std::ranges::generate_random` and in the loop. But we are forced to allocate a temporary storage for the random numbers. Given the size of `npath`, we run into L1 or L2 cache misses with sub-optimal performance at the end.

Let us apply bufferization:

```
std::mt19937 engine(777);
std::normal_distribution distribution(0., 1.);
std::array<double, nbuffer> rand; // e.g., nbuffer=128

double v0 = 0, v1 = 0;

for(std::size_t p = 0; p < npath; p += nbuffer) {
    std::size_t local_size = (p + nbuffer <= npath) ? nbuffer : (npath - p)

    std::ranges::generate_random(std::span(rand.begin(), local_size), engir

    for(std::size_t b = 0; b < local_size; ++b) {
        double res = std::max(0., S * exp(sqrt(T) * vol * rand[p] + T * mu)
        v0 += res;
        v1 += res * res;
    }
}

result     = v0 / npath;
confidence = v1 / npath;
```

The code is more complex, but is likely faster, and allows one to tune its performance by changing the `nbuffer` parameter.

§ 2.2. Low-level API with `std::simd`

Let's investigate the opportunities for low-level SIMD based API in the same benchmark. The API for SIMD-level RNG is not defined yet, but there are several possible approaches with their own pros and cons, which we may consider. The key distinguishing property in these approaches is the level at which the API gets the information about `std::simd` type being requested by the user. An engine, pseudo-random number generator that produces integer sequence with uniform distribution, holds the state for the RNG generation including extra intermediate structures for efficient calculation. With the knowledge about SIMD size, it may store internal structures in the most efficient format and this format does differ in practice for different SIMD sizes.

In the ideal world, distribution objects should be almost stateless (except for the parameters of the distribution), but in practice they are not. For example, Box-Muller method for Gaussian random

number generation generates two variates using two independent samples from the uniform distribution. Thus, in the scalar use one already generated value can be stored in the distribution object to be returned on the next call.

This results in an amusing corner-case:

```
std::mt19937 E1(777);
std::mt19937 E2(777);
std::normal_distribution D(0., 1.); // Box-Muller
for(;;) {
    double res1 = D(E1);
    double res2 = D(E2);
}
```

One may naively expect that `res1 == res2`, because the values are expected to be consumed from 2 engines in the same starting state. But in fact all numbers will be generated using `E1`, while the `E2` engine will never be used in the most existing C++ standard library implementations. The current standard wording specifies the output of a distribution object when used with "successive invocations with the same [engine] objects", thus the result above is unspecified and observed behavior is valid.

§ 2.2.1. Engine template parameter

In our first example, both engine and distribution will be constructed with the knowledge about the `std::simd` type, which will be requested from them.

```
std::mt19937<std::fixed_size_simd<std::uint_fast32_t, 16>> E(777);
std::normal_distribution<std::fixed_size_simd<double, 16>> D(0., 1.);
auto rand = D(E);
```

`std::simd` type in the engine and `std::simd` type in the distribution are different. The number of values produced by the engine per value required by distribution is currently unspecified in the standard (even in scalar case), but in most existing implementations 2 values of `std::uint_fast32_t` will be consumed from the engine per double return result of the `normal_distribution`.

Implementation wide it may (or may not) be more optimal to define it as:

```
// 32 SIMD size passed to engine
std::mt19937<std::fixed_size_simd<std::uint_fast32_t, 32>> E(777);
// 16 SIMD size passed to distribution
std::normal_distribution<std::fixed_size_simd<double, 16>> D(0., 1.);
auto rand = D(E);
```

Going towards more sophisticated distributions acceptance-rejection type of methods are becoming more common, which consume a varying number of engine values per resulting value.

Thus, there is no universal good answer for the proper `std::simd` size of the engine output, which would be the best fit for the given distribution `std::simd` output.

If engine is capable of generating values only in the packs of fixed size (defined by the type of the simd parameter) and distribution consumed this pack only partially, then the question about the use of the remaining numbers is raised:

1. Remaining part can be stored in the distribution object in either raw engine values form or in the form of already produced, but not returned to the application values. That brings two more oddities:
    1. Distribution is unaware of the pack size returned from the engine at construction point - it gets the engine type in `operator()`. The size of potential tail is unknown thus static allocation of the storage for it is problematic.
    2. It extends the corner-case example above into a more complicated form, where it is unclear what was actually the source of the randomness - current function argument or something from the past.
2. Remaining part can be discarded. While not entirely thrifty, it might not be that bad choice.
    1. NOTE: There is no solid reason for distribution to know about `std::simd` type at construction point in this case, except for providing corresponding member function in the API

Having all those implementation implications in mind, let us look at our example:

```
constexpr std::size_t size = 16;
std::mt19937<std::fixed_size_simd<std::uint_fast32_t, size>> E(777);
std::normal_distribution<std::fixed_size_simd<double, size>> D(0., 1.);
```

```
double v0 = 0, v1 = 0;
std::size_t p = 0;

for(; p + size <= npath; p += size) {
    auto rand = D(E); // std::fixed_size_simd<double, size>
    auto res = std::max(0., S * exp(sqrt(T) * vol * rand + T * mu) - X);
    v0 += std::reduce(res);
    v1 += std::reduce(res * res);
}

// npath is runtime parameter - it may not be divisible by 16
if (p != npath) {
    auto rand_tail = D(E);
    auto res = std::max(0., S * exp(sqrt(T) * vol * rand_tail + T * mu) - X
    for(std::size_t i = 0; p + i < npath; ++i) {
        v0 += res[i];
        v1 += res[i] * res[i];
    }
    // discarding unused part of res
}
result     = v0 / npath;
confidence = v1 / npath;
```

If `npath` is not a multiple of `std::simd` size, then this example generates more random numbers than actually needed in `rand_tail` and throws away unused part. That makes the example not an absolute match to our initial example, but it is as close as we can get.

There is also an open question what `E.discard(1)` means in this case - whether it discards a `std::simd` pack of 16 values or only one element.

> NOTE: We do not want to combine the logic of the tail with the main loop in order to make our hot path as optimized as possible.

2.2.2. Engine template parameter + rebind constructor

One may overcome the main limitation of the approach by allowing rebind construction of the engine with the different `std::simd` widths and scalar type.

```
constexpr std::size_t size = 16;
std::mt19937<std::fixed_size_simd<std::uint_fast32_t, size>> E(777);
std::normal_distribution<std::fixed_size_simd<double, size>> D(0., 1.);

double v0 = 0, v1 = 0;
std::size_t p = 0;

for (; p + size <= npath; p += size) {
    auto rand = D(E);
    auto res = std::max(0., S*exp(sqrt(T) * vol * rand + T * mu)-X);
    v0 += std::reduce(res);
    v1 += std::reduce(res * res);
}

if (p != npath) {
    std::mt19937 E_tail(E); // rebinding to scalar type
    std::normal_distribution D_tail(0., 1.); // getting scalar distributior
    for(; p < npath; ++p) {
        auto rand_tail = D_tail(E_tail);
        auto res = std::max(0., S * exp(sqrt(T) * vol * rand_tail + T * mu)
        v0 += res;
        v1 += res * res;
    }
    E = E_tail; // rebinding back to keep original engine in the expected s
}

result     = v0 / npath;
confidence = v1 / npath;
```

While user-level flexibility is obtained, a set of redundant copy operations were introduced into the code, which will bring visible overheads especially in case of engines with a larger state.

§ 2.2.3. Algorithm-like function template parameter

An alternative solution would be to shift the knowledge of the `std::simd` size from engine/distribution construction point to the point of usage. This makes the engine unaware of its main usage mode and thus internal layout would be chosen by implementation to some balanced form, which allows both packed and scalar generation.

```
std::mt19937             E(777);
std::normal_distribution D(0., 1.);
auto rand = std::generate_random_simd<std::fixed_size_simd<double, 16>>(E,
```

While it limits bare metal optimizations of the engine layout, it provides a significant freedom of implementation to the standard library - the way of consuming base random numbers out of an engine may vary from platform to platform. It also brings in more flexibility on the usage side.

```
constexpr std::size_t size = 16;
std::mt19937 E(777);
std::normal_distribution D(0., 1.);

double v0 = 0, v1 = 0;
std::size_t p = 0;

for (; p+size <= npath; p += size) {
    auto rand = std::get_random_simd<std::fixed_size_simd<double, size>>(E,
    auto res = std::max(0., S * exp(sqrt(T) * vol * rand + T * mu) - X);
    v0 += std::reduce(res);
    v1 += std::reduce(res * res);
}

// scalar tail matches original example implementation
for (; p < npath; ++p) {
    auto rand_tail = D(E);
    auto res = std::max(0., S * exp(sqrt(T) * vol * rand_tail + T * mu) - X
    v0 += res;
    v1 += res * res;
}

result     = v0 / npath;
confidence = v1 / npath;
```

This example matches the initial example exactly and the user is not forced to copy engine state to deal with tails. Moreover, having some belief in performance outcome, one may modify example to optimize the tail computation more, by generating a sequence of decreasing `std::simd` sizes of 8, 4, 2 and 1.

§ 3. SIMD-level API vs High-level API

There is no single answer on whether High-level or SIMD-level API should be used to optimize a given scalar code with random number generation.

High-level API requires additional buffers and may need additional blocking to achieve good performance.

Low-level API may provide top performance but at the same time, SIMD-level API brings in a number of implications on the user code - more efforts for dealing with tails, more code restructuring to make all surrounding code `std::simd` fashioned.

At the same time any discussed Low-level API can be used under the hood of higher level API.

1. Engine template parameter approach will require more API tuning under the hood of High-level API. Some kind of rebind operation will be needed to transform incoming legacy engine into another one with `std::simd` template parameter.
2. Algorithm-like approach can be seamlessly used without additional modifications.

With that we believe that both API levels make sense for the standard library.

§ 4. Summary

With the analysis of RNG API for support of vectorization, we conclude that:

1. High level API aims at the support of the majority of the RNG based applications created by a regular C++ developer. With opportunity to enable HW vectorization capability features by vendors under the hood of the API implementation it would further enhance performance of C++ RNG based applications. The blocking techniques for the further improvement of the performance are also possible upon the same API. The implementation of the API might rely on SIMD capability.
2. Low-level API aims at the advanced developers who need to code closer to HW still staying in C++ convention. Though, the definition of SIMD based RNG API should avoid making the

developers understand the computational SIMD related specifics of RNG based on engine and distribution concepts. The key to that might be a single point of SIMD configuration in the API, for example, a SIMD-parameterized generation function. How this approach correlates with the SIMD use in other domains, for example, mathematical functions, - open question.

Based on those observations having both high- and low-level vectorized API for RNG in the C++ standard library looks reasonable.

References

Informative References

[P1068R7]

Ilya Burylov; et al. *Vector API for random number generation*. URL: http://wg21.link/P1068R7

[P1928R3]

Matthias Kretz. *Merge data-parallel types from the Parallelism TS 2*. 3 February 2023. URL: https://wg21.link/p1928r3